

Pace

Farhan Kalam
May 20, 2026

Abstract

Pace is a conversational transit assistant for the Massachusetts Bay Transportation Authority (MBTA). It tells you *when* to leave and *how risky* the trip is, not just how long it takes. Ask it a question and Pace returns a grounded answer with cited sources, a recommended departure time, and a refusal when the data isn't there. Snap a photo of a Boston or Cambridge parking sign and Pace tells you if you can park there right now, factoring in the current date, time, and which city's rules apply. Over time, Pace learns each user's usual commute and proactively notifies when a disruption affects it. This document summarizes the v1 scope, architecture, timeline, evaluation strategy, and key design decisions for the project.

1 Overview

Boston-area transit information is scattered across MBTA alerts, boston.gov parking PDFs, Cambridge ordinances, Bluebikes, and weather. Tools like Google Maps give point estimates of trip duration; they do not reason about risk, deadlines, parking rules, or current disruptions. Pace adds a reasoning layer over public data, exposed through a chat interface, a map view, and proactive push notifications.

What makes Pace different.

- Recommends a departure time, not just trip duration — factoring in deadlines, current alerts, weather, and historical line reliability at that hour.
- Calibrated risk labels (low / mid / high) on every routing answer, measured by Expected Calibration Error.
- Reads Boston and Cambridge parking signs from a photo and reasons about the current date, time, and which city's rules apply.
- Reconciles permit zones with the user's location so "Resident Only Zone B5" is answered against where the user actually is.
- Learns each user's commute pattern and proactively pings when a disruption affects the usual route.
- Cites sources for every claim and refuses when the data is insufficient.

v1 scope. Hybrid retrieval over transit, Boston and Cambridge parking, and bike data; live MBTA alerts at retrieval time; a multi-modal route planner with recommended departure times, fare, accessibility, and bike availability; pre-computed backup routes for common origin-destination pairs; a delay-prediction model behind calibrated risk labels; the parking-sign photo reader with permit-zone reconciliation; map view with drawn routes; strict citation-only answers; last/first train, multi-stop trips, and calendar (.ics) export; device-keyed personal memory with learned routines, saved places, transparency, and one-click data delete; proactive push notifications through a Progressive Web App; onboarding, settings, and query history.

2 Architecture

Pace is split into a Next.js frontend, a FastAPI backend, and a Postgres with pgvector data store backed by Redis caching.

2.1 Request pipeline

Each query moves through a small set of stages.

1. **Intent classification** routes the query into one of six domains: route, alert, parking-rules, parking-sign, schedule, or off-topic.
2. **Domain-routed retrieval** runs hybrid BM25 plus dense search via pgvector over only the relevant index, fused with reciprocal rank fusion, then re-ranked by a cross-encoder.
3. **Specialist subsystems** run as needed: the multi-modal route planner, the delay-prediction model for risk-aware scheduling, and a vision-language model for parking signs.
4. **Generation** happens through a single LLM (`gpt-4o-mini`) at temperature zero. The system prompt requires every claim to cite a retrieved chunk.
5. **Confidence gate** refuses to answer when no retrieved chunk passes the score threshold, returning an explanation rather than guessing.

Responses stream to the client via Server-Sent Events. Location, when granted, is passed alongside the query as the default origin for transit and parking-sign answers.

2.2 Data layer

Eight sources feed the index: MBTA V3 (schedules, alerts, fares, elevators), MBTA LAMP for historical performance, boston.gov for parking PDFs and street-cleaning CSVs, Cambridge Open Data via the Socrata API, Cambridge Title 10 ordinances from Municode, permit zone polygons (Boston resident zones and Cambridge visitor zones), GBFS for Bluebikes with live availability, and Open-Meteo for weather. Refresh cadence ranges from live per-request (alerts, weather, Bluebikes) through daily (MBTA schedules) to weekly (boston.gov PDFs, Cambridge Open Data) and monthly (Title 10 ordinances, permit zones). Chunks land in Postgres with pgvector embeddings produced by BGE-small at 384 dimensions: routes and stops become one chunk per stop with route, accessibility, and fare attached; ordinances become 300-word passages; CSVs become one chunk per street segment with the day rule; permit zones become one chunk per polygon with the restriction text.

2.3 ML components

- **Retrieval.** BM25 for lexical, pgvector cosine for dense, fused via reciprocal rank fusion ($k = 60$), re-ranked with `bge-reranker-base` on the top 20 to pick the top 5.
- **Generation.** `gpt-4o-mini` for all queries. Temperature zero; citation-only system prompt.
- **Delay prediction.** XGBoost regressor on tabular features (line, station, hour-of-day, day-of-week, weather, active alerts, historical mean delay for that slot) with isotonic calibration. Trained on a time-based split (last four weeks held out) to avoid future-information leakage. Risk thresholds: $P(\text{delay} > 10 \text{ min})$ under 0.2 = low, 0.2–0.5 = mid, over 0.5 = high.
- **Vision.** OpenAI vision starting with `gpt-4o-mini`; falls back to `gpt-4o` only if accuracy on the 50-photo sign eval set drops below the 0.85 target.

3 Timeline

The build runs thirteen weeks across four phases.

Phase	Focus
Weeks 1–3	Lock scope, write the eval seed set, build data pipelines (MBTA, boston.gov, Cambridge), implement hybrid retrieval and the evaluation harness with a baseline comparison.
Weeks 4–6	End-to-end Q&A with the confidence gate; fare, accessibility, bike, and last-train surfaces; train the delay-prediction model with isotonic calibration; build the multi-modal route planner with multi-stop and future-time support.
Weeks 7–10	Parking sign VLM; frontend chat, map, onboarding, settings, history, multi-stop input, calendar export; personal memory with device identity, saved places, transparency, and data delete; notifications via PWA install, permission flow, and the alert-matching worker.
Weeks 11–13	Buffer to absorb scope debt and tune metrics; the 30-query Pace Test versus <code>gpt-4o-raw</code> ; deploy and soft-launch; write the case study and produce the demo.

4 Evaluation Strategy

Quality claims rest on a held-out evaluation set committed to the repository. The set is authored in Week 1 with thirty seed queries and expands to two hundred by Week 11. Its distribution matches expected real usage rather than uniform stratification: routing dominates (35%), followed by alerts (20%), parking-rules and schedule-aware questions (15% each), parking signs (10%), and a small off-topic slice (5%). Within each domain, writing styles mix as 50% casual, 25% formal, 15% typos or abbreviations, and 10% adversarial. Every commit can re-run the harness against a fixed baseline (`gpt-4o-raw` with no retrieval) so regressions surface immediately. A separate holdout set is run sparingly for the README’s final numbers; tuning against it invalidates the metric.

Metric	Target	What it measures
Recall@1	> 0.75	top retrieval result is correct
Recall@5	> 0.90	retrieval finds the right document
Faithfulness (RAGAS)	> 0.90	answers stay within cited sources
Answer correctness	> 0.85	answer matches ground truth
Hallucination rate	< 3%	citation enforcement holds
Refusal rate	5–10%	honest refusal when sources are weak
Per-domain accuracy (min)	> 0.80	every intent class above floor
Risk calibration (ECE)	< 0.10	“high risk” actually means high risk
Parking sign accuracy	> 0.85	VLM reasoning is reliable
Data freshness (alerts)	< 60 s	live alerts feel live
Route planning latency	< 500 ms	backend routing isn’t a bottleneck
TTFT (streaming)	< 800 ms	response starts feeling fast
p95 latency (text)	< 2 s	end-to-end feels responsive
p95 latency (VLM)	< 6 s	sign reading is acceptable
Text cost per query	< \$0.005	mini-only keeps unit cost low
VLM cost per query	< \$0.03	parking sign costs bounded
Notification precision	> 0.85	fired notifications correspond to real disruptions
Notification recall	> 0.80	real disruptions on a routine actually fire a notification

5 Design Decisions

The most consequential choices, in brief.

Citation-only answers. Every claim traces to a retrieved chunk. When retrieval is insufficient, the system refuses with an explanation instead of guessing. Expected refusal rate is 5–10%. This is what separates Pace from a chatbot that confidently makes things up.

Time-based train/test split. Random shuffling on time-series data leaks future information and turns calibration claims into fiction. The last four weeks of MBTA performance data are held out; everything prior trains the delay model.

\$50 total summer budget. Total OpenAI usage is capped at \$50 for the project, enforced two ways: an OpenAI dashboard monthly cap at approximately \$15 with alerts at 50, 80, and 100 percent, and cost-guard middleware that refuses LLM calls once the projected total would exceed \$50.

Device-keyed identity, not traditional accounts. A UUID generated client-side on first visit, stored in `localStorage`, sent as an `X-Device-Id` header. No email or password. Skips the auth and compliance work while still enabling per-user routines. Privacy-friendly — no PII collected.

Personal memory and notifications in v1. Routine learning, saved places, “What Pace knows” transparency, one-click delete, and Web Push notifications when MBTA alerts affect stored routines.

Permission asked only after a positive interaction. This is the retention loop that turns Pace from an occasional tool into an app worth opening daily.

PWA as the launch surface. A Progressive Web App delivers most of the native experience at zero cost. iOS Web Push works only when installed to the home screen, which the install prompt handles. Native iOS and Android are deferred to v2.

Boston and Cambridge for parking; transit covers the full MBTA. Rule-based parking answers cover Boston and Cambridge only — Cambridge is the second-densest MBTA-area municipality, and excluding it would make the coverage claim misleading. Transit answers extend to the rest of the MBTA service area. Other municipalities are deferred to v2.

Single model in v1. Text uses `gpt-4o-mini` only. Vision starts with `gpt-4o-mini` and falls back to `gpt-4o` only if eval accuracy drops below the 0.85 target. No Anthropic provider, no complexity router. One vendor, one dashboard, one key.

6 Stack and Layout

Layer	Choice
Frontend	Next.js 14, TypeScript, Tailwind, shadcn/ui, Leaflet, PWA (manifest + service worker)
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic, LangGraph (tentative), pytest
Storage	Postgres + pgvector, Redis
ML	BGE embeddings, BM25, BGE re-ranker, GPT-4o-mini, OpenAI vision, XG-Boost, RAGAS
Data	MBTA V3 and LAMP, boston.gov, Cambridge Open Data, Cambridge Title 10, permit zone polygons, GBFS, Open-Meteo
Push	Web Push (VAPID) and Apple Web Push via PWA
Hosting	Vercel (frontend), Railway (backend), Neon (database)

The repository is organized around clear separations: `frontend/`, `backend/`, `data/` for scrapers and ETL, `models/` for training and artifacts, `prompts/` for versioned system prompts, `eval/` for the test set and harness, and `infra/` for deployment configs. These folders are planned but not yet created; each is initialized in the week it is needed.

7 Closing

Pace is intentionally a focused, measurable system rather than a feature checklist. The bet is that depth on a handful of components — grounded retrieval, calibrated risk prediction, honest refusal, real route planning, and trustworthy personalization — produces a more useful and more credible artifact than a wider but shallower app would. The metrics in `eval/`, not feature counts, are the proof.